book is that it's currently the de facto standard framework for contract testing, especially for teams who want to adopt CDCT.

> **NOTE** PDCT isn't a replacement for CDCT, but rather it helps teams to adopt contract testing easily, especially if they can't use CDCT.

Even with the numerous benefits, we're aware that there are still drawbacks with CDCT, especially when it comes to adoption. As first mentioned in chapter 2, section 2.5, CDCT has the following disadvantages:

- Takes time
- Requires a mindset shift
- Requires a lot of buy-in
- More technical than other testing activities
- Difficult to do when external teams are involved

There are also other drawbacks because the CDCT approach isn't suited for API-first design workflows. In a consumer-driven approach, the provider must wait for the consumer to publish the contract first. What if this approach doesn't suit your development and testing needs?

In the following subsections, we explain in more detail why PDCT was introduced, how PDCT addresses all the drawbacks of CDCT, and some tradeoffs to be aware of that also come with PDCT. By the end of this chapter, you really should have no excuse for not trying out contract testing!

11.1.1 Why was PDCT introduced?

Over time, there has been a big increase in how much CDCT is used, especially as more and more people starting using microservices. This way of testing is popular because it helps developers feel more confident about their work, makes deployments happen faster, cuts down on problems when things go live, and saves money on testing. Lots of companies, no matter how big or small, are swapping out their older, end-to-end tests for